



Programme

Master of Software Engineering

Course

**MSE800 Professional Software Engineering
(Level 8, 30 Credits)**

Assignment 1

**Object-Oriented Programming
Assignment**

(Classification - Individual)

Car Rental Management System Design, Architecture, Testing, and Evolution Report

Course: MSE800 Professional Software Engineering (Level 8)

Assessment: Assignment 1 – Object-Oriented Programming

Student: Wijemuni Rukshan Madushanka De Silva

Student ID: 270802602

Programme: Master of Software Engineering

Table of Contents

1. Abstract.....	5
2. Introduction.....	5
2.1 Background and Problem Context.....	5
2.2 Objectives and Scope	5
2.2.1 System Features (Version 2).....	5
2.3 Assumptions and Constraints	6
3. Requirements Analysis.....	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements.....	9
3.3 User Roles and Permissions.....	10
Functional Requirements Explanation	10
4. System Design and Architecture (Task 1 – LO1).....	10
4.1 Architectural Style	10
Component Diagram	11
4.2 Design Patterns and Justification	11
Singleton Pattern (RentalSystem)	11
Factory Method Pattern (UserFactory)	11
4.3 UML and Data Modelling Diagrams.....	11
4.4 UML Diagrams and Explanation (Task 1 – Design & Architecture).....	12
4.4.1 Class Diagram	13
4.4.2 Use Case Diagram.....	14
4.4.3 Sequence Diagram – Booking Flow	15
4.4.4. Entity Relationship (ER) Diagram	16
4.4.5 UML Summary.....	17
5. Implementation and Code Walkthrough.....	17
5.1 Project Modules and Responsibilities.....	17
5.2 Application Entry Point (main.py)	18
5.3 Step-by-Step Example: Customer Booking Flow	19
5.4 Core Business Logic (system.py).....	21

5.5 Object Models (models.py).....	22
5.6 Database Layer (database.py).....	23
5.7 Data Persistence Rationale (SQLite)	24
6. Deployment and Release Build.....	24
6.1 Deployment Environment	24
6.2 Build and Execution Process	24
6.3 Release Management	25
7. User Documentation	25
7.1 User Roles	25
7.2 Admin Workflow.....	25
7.3 Customer Workflow.....	26
7.4 Error Handling and Feedback.....	27
8. Testing Strategy and Test Cases	27
8.1 Testing Approach	27
8.2 Test Cases (Traceable to Requirements).....	27
8.3 Coverage Discussion.....	29
9. Innovative Solution (Task 2 – LO2)	30
9.1 Industry Need	30
9.2 Proposed Innovation: AI-Driven Demand Forecasting for Dynamic Pricing.....	30
9.3 Architectural Support.....	30
10. Software Evolution, Maintenance, and Backward Compatibility (Task 3 – LO2)..	30
10.1 Maintenance Plan.....	30
Corrective Maintenance	31
Adaptive Maintenance	31
Perfective Maintenance.....	31
Backward Compatibility Strategy	31
10.2 Versioning Strategy.....	32
10.3 Versioning History	32
Version 1 – Baseline Implementation.....	32
Version 2 – Enhanced Intelligent System	32
10.4 Backward Compatibility Strategy	32
11. User Documentation Summary.....	32
12. Limitations and Future Work.....	33

Known Limitations.....	33
Future Enhancements.....	33
13. Conclusion.....	33
14. References	34

Table of Figures

Figure 1. Component Diagram.....	11
Figure 2. Class Diagram.....	13
Figure 3. Use Case Diagram	14
Figure 4. Sequence Diagram	15
Figure 5. ER Diagram	16
Figure 6. Application control flow	19
Figure 7(a): Main application loop and authentication menu logic implemented in main.py	20
Figure 8. RentalSystem class implementation (Singleton pattern)	22
Figure 9. SQLite table creation logic	23
Figure 10. Admin menu options displayed in the CUI	26
Figure 11. Customer booking and recommendation output.....	26
Table 1. Functional Requirements Table	7
Table 2. Non-Functional Requirements Table	9
Table 3. User Roles and Permissions Table	10
Table 4. Project Modules and Responsibilities Table	17
Table 5. Object Models Table	22
Table 6. Deployment Environment Table	24
Table 7. User Roles Table	25
Table 8. Test Cases Table	27

1. Abstract

This report presents the design and implementation process for an automated Car Rental Management System developed using the Python programming language. The system eliminates the need for traditional car rental management techniques and incorporates a highly efficient command-line-based system with access control for both Admin and Customer types of users. SQLite has also been integrated to satisfy the condition of not manually configuring the environment for the system, as required for the assessment. The design reflects highly advanced Object-Oriented Programming (OOP) concepts and incorporates two appropriate design patterns, namely the Singleton pattern for system coordination and the Factory Method pattern for user role creation. The report also presents the analysis, design, implementation, testing with traceable test cases, an industry-related innovation proposal, and the evolution plan for the developed software.

2. Introduction

2.1 Background and Problem Context

Currently, most small car rental companies use manual methods such as paperwork or spreadsheets to manage their vehicles and bookings. This system is not reliable; for example, there may be cases where there is a double booking, wrong fee calculations, and so on, and there may be problems with keeping track of the vehicle's availability in real time. This project will make these processes more reliable by automating these tasks.

In Version 2, several innovations, which are more suitable for the industry, have been incorporated into the booking system, such as dynamic long-term discounts, pricing explanations, alternative vehicles for booking failure, and risk-related decision support for Admin approvals.

Below, there is a suggestion for another innovation, which may be built on top of the existing system, that is, persisted data, for the future: AI-based demand forecasting.

2.2 Objectives and Scope

2.2.1 System Features (Version 2)

The Car Rental Management System provides the following functionality:

- Role-based authentication for Admin and Customer users

- Admin CRUD operations for managing vehicle inventory
- Customer browsing and booking of available vehicles
- Booking approval and rejection workflow managed by Admins
- Automated validation of rental duration constraints
- Dynamic pricing with long-term rental discounts (5% for ≥ 7 days, 10% for ≥ 14 days)
- Explainable pricing breakdown showing base cost, discount, and final payable amount
- Smart alternative car recommendations when booking conditions fail
- Risk-aware booking management, displaying customer risk levels to Admins
- SQLite-based persistence for users and vehicles, restored at system startup
- Comprehensive logging of system activity for traceability and debugging

These enhancements reflect real-world rental platform behaviour and significantly improve usability, transparency, and decision support.

The complete source code and version history for this project are publicly available on GitHub at the link provided below.

[GitHub Repository](#)

2.3 Assumptions and Constraints

- The application is operated via a command-line interface (CUI).
- Admins use the provided secret key for admin registration.
- The system runs locally and stores data in an SQLite database file in the project's database/ directory.
- No external dependencies beyond the Python standard library are required.

Assumptions:

1. The system is operated by users with basic computer literacy.
2. All rentals are handled within a single operational context.
3. Payment processing is handled externally.
4. Users provide valid input.

Constraints:

1. CLI-based Python system only.
2. Standard libraries only.
3. No manual configuration required.
4. Academic time constraints apply.

Stakeholders:

1. Administrators
2. Customers
3. Academic Assessors
4. Developers

3. Requirements Analysis

3.1 Functional Requirements

Table 1. Functional Requirements Table

ID	Requirement Description	Priority
FR1	User Management: Secure registration and login for Admin and Customer roles with role-based access control.	High
FR2	Car Management: Admin CRUD for vehicles including mileage, rental limits, pricing, and availability.	High
FR3	Rental Booking: Customers view available vehicles and request bookings for a duration.	High
FR4	Dynamic Fee Calculation: Automatic calculation based on daily rate, rental days, and	High

	applicable long-term discounts.	
FR5	Booking Management: Admins approve or reject customer booking requests and update car availability accordingly.	High
FR6	Pricing Transparency: System displays a clear pricing breakdown (base cost, discount applied, final total).	High
FR7	Alternative Vehicle Recommendation: When a booking fails due to availability or rental constraints, the system suggests suitable available alternatives.	Medium
FR8	Risk-Aware Decision Support: Admins can view customer risk indicators while managing bookings.	Medium
FR9	Data Persistence: SQLite storage restores user and vehicle records after restart (bookings remain in-memory in this submission).	High

Functional requirements are used to describe the system's core capabilities. The system's requirements were refined to address intelligent pricing, recommendation logic, risk-aware decision support, and partial persistence in Version 2.

3.2 Non-Functional Requirements

Table 2. Non-Functional Requirements Table

Category	Requirement	Explanation
Security	Role-Based Access Control (RBAC)	Customers cannot access Admin vehicle/booking management features; Admin-only actions are gated by role.
Reliability	Robust input validation	Non-numeric, invalid, and out-of-range inputs are handled without crashing; users are reprompted safely.
Usability	Clear menu-driven CUI	Role-specific menus provide clear prompts, feedback, and error guidance for each action.
Transparency	Explainable pricing output	Customers receive a breakdown of base cost, discount (if applicable), and final payable amount.
Maintainability	Modular architecture	Separate modules for UI, business logic, models, and persistence functions support maintainability and testing.
Scalability	Extensible design	Architecture supports future enhancements (e.g., full persistence, analytics, forecasting) without major refactoring.
Deployability	No manual configuration	Logs directory and SQLite database file are auto-created;

		no external services or packages required.
Data Integrity	Persistent state restoration	User and vehicle data are restored from SQLite on startup to maintain continuity across sessions.

3.3 User Roles and Permissions

Table 3. User Roles and Permissions Table

Role	Permissions	Reasoning
Admin	Add/Update/Remove cars; View vehicles; Approve/Reject bookings	Administrative control over inventory and approvals.
Customer	View available cars; Create booking requests; View own booking status/history	Limited access aligned to customer actions.

Functional Requirements Explanation

Functional requirements define the system’s core capabilities. Each requirement is implemented directly in the system logic and validated via test cases. The table summarizes these requirements, while this section explains their relevance.

4. System Design and Architecture (Task 1 – LO1)

4.1 Architectural Style

Layered architecture is used to ensure separation of concerns. The Presentation Layer is defined in `main.py`, the Business Logic Layer in `src/system.py`, where rental rules are enforced, and the workflow is orchestrated through the use of model entities defined in `src/models.py`. The Data Access Layer consists of SQLite persistence, which is integrated into the initialization of the system. This separation of concerns makes it easier to test the code due to the well-defined responsibilities of each layer.

Component Diagram

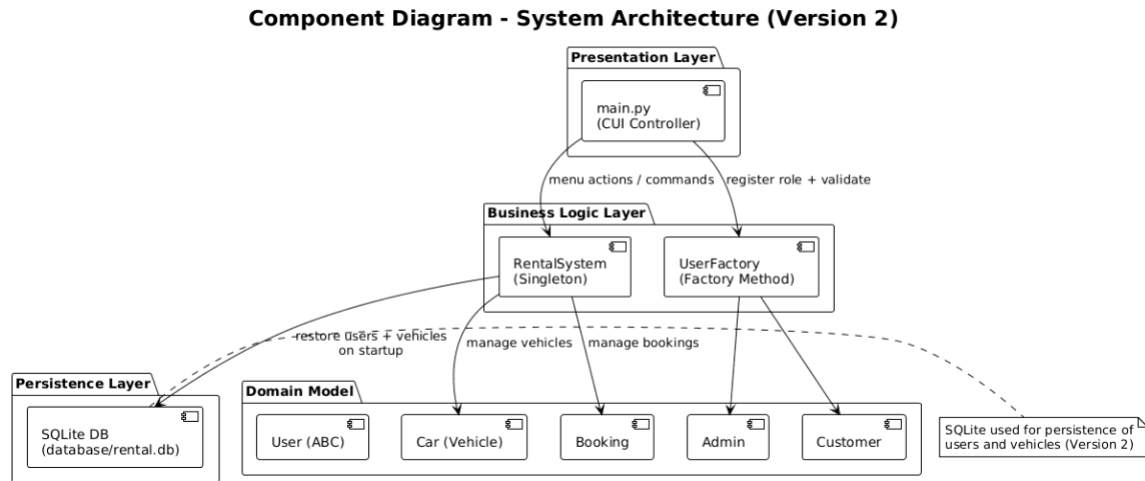


Figure 1. Component Diagram

4.2 Design Patterns and Justification

Singleton Pattern (RentalSystem)

`RentalSystem` is implemented as a Singleton to ensure a single source of truth for cars, users, bookings, and the database connection. This prevents inconsistent states and mirrors real-world systems where a central service coordinates business operations.

Factory Method Pattern (UserFactory)

`UserFactory` centralizes the creation of `Admin` and `Customer` objects. This improves security and maintainability by validating the role string and enforcing an Admin Secret for admin registration. The pattern also reduces duplication because role creation logic exists in one place.

4.3 UML and Data Modelling Diagrams

UML and data modelling diagrams are provided in the separate *Design and Architecture* PDF. This report explains how to interpret these diagrams and how they relate to the implemented system design:

- **Class Diagram:** Illustrates the static object-oriented structure of the system, showing relationships between the abstract User class and its subclasses (Admin and Customer), as well as associations with Car, Booking, and the RentalSystem Singleton.
- **Use Case Diagram:** Represents functional interactions between system actors, clearly differentiating **Admin** and **Customer** capabilities such as vehicle management (CRUD), booking requests, and booking approvals, enforcing role-based access control.
- **Sequence Diagram:** Demonstrates the dynamic behavior of the system during the booking workflow, tracing the interaction from a customer's booking request through validation, persistence, admin approval or rejection, and the subsequent update of car availability.
- **Entity Relationship (ER) Diagram:** Models the underlying data structure of the system, defining the User, Car, and Booking entities and their one-to-many relationships. The ER diagram complements the class diagram by focusing on data persistence and referential integrity within the SQLite database, supporting scalable and consistent storage.
-

4.4 UML Diagrams and Explanation (Task 1 – Design & Architecture)

This section presents and explains the UML diagrams used to model the structure and behavior of the Car Rental Management System. The diagrams visually represent the object-oriented design, role-based interactions, and execution flow, supporting clarity, maintainability, and alignment with software engineering best practices.

4.4.1 Class Diagram

Class Diagram - Car Rental Management System (Version 2)

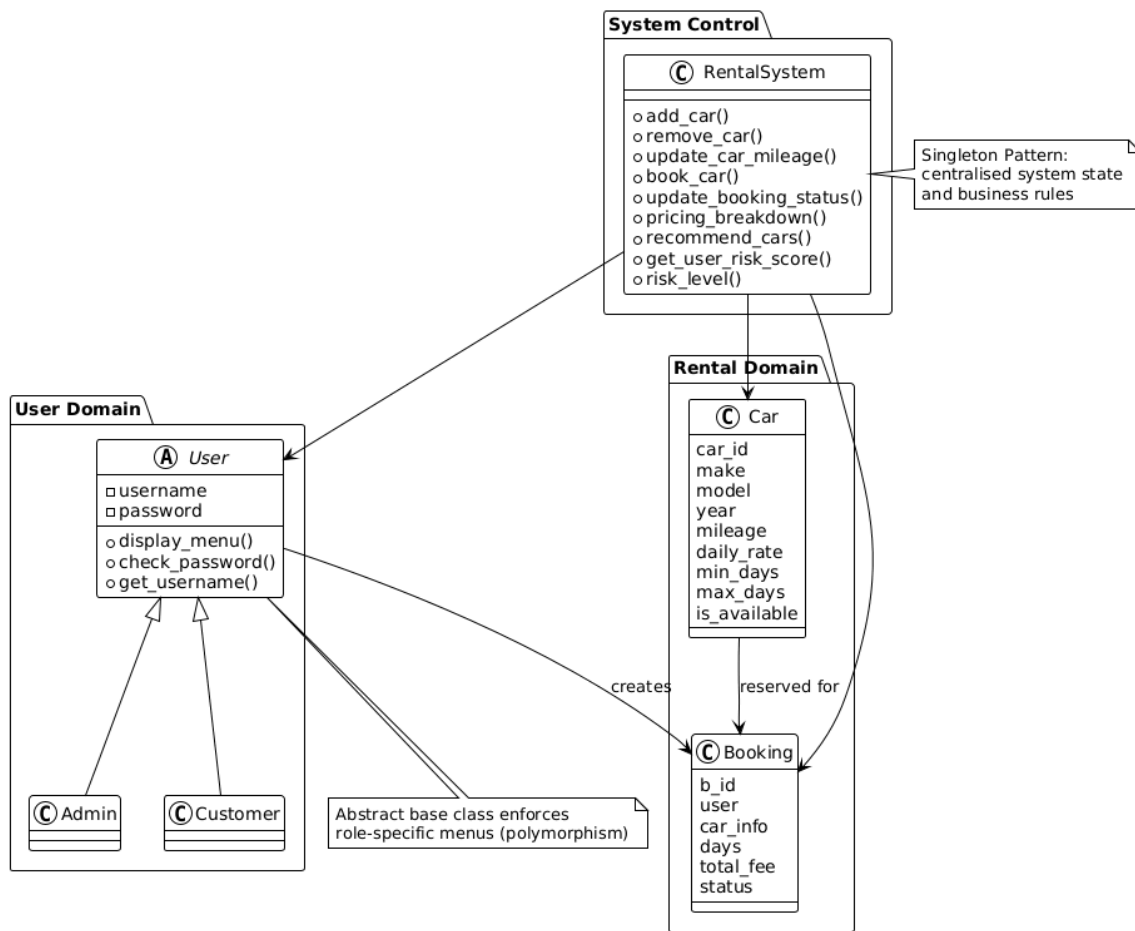


Figure 2. Class Diagram

The Class Diagram represents the static view of the system's structure. It also shows how Object-Oriented Programming concepts have been applied. The Rental System's static structure is defined by the 'RentalSystem' class. It's a Singleton, which guarantees a single source of truth for users, cars, bookings, and database connections. The 'User' class is an abstract data type. All users implement common authentication logic. The 'Admin' and 'Customer' classes inherit from 'User' and override methods for their respective roles' menu functionality. 'Car' and 'Booking' classes represent the system's main data models. 'UserFactory' applies the Factory Method concept for user creation. 'Database' represents data access using SQLite.

4.4.2 Use Case Diagram

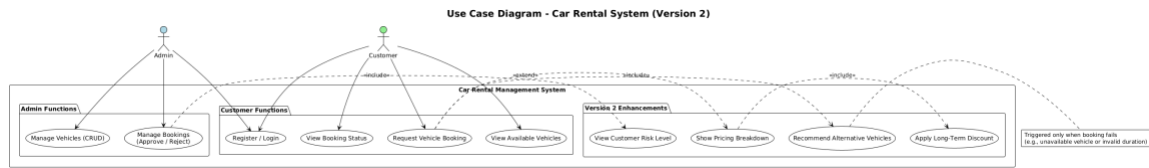


Figure 3. Use Case Diagram

The Use Case Diagram shows the functional interactions between the Admin and Customer roles and the Car Rental Management System. The Customers have limited functionality for viewing cars, booking cars, and viewing booking status, whereas the admin role has the privilege for managing vehicles and approving bookings. This directly maps to the functional requirements as identified during the requirements analysis.

4.4.3 Sequence Diagram – Booking Flow

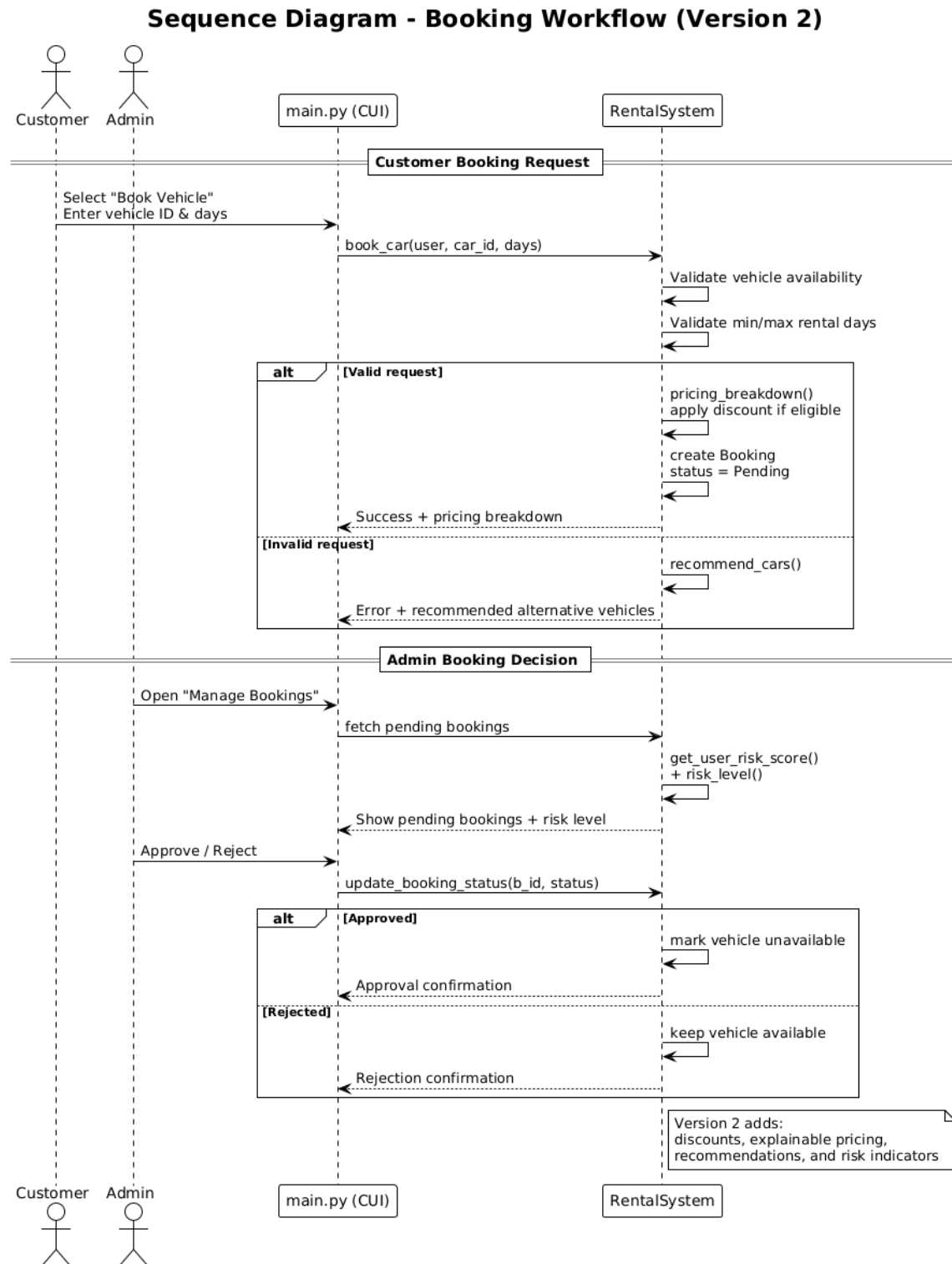


Figure 4. Sequence Diagram

The Sequence Diagram illustrates the dynamic behavior of the system during the end-to-end booking process. It depicts how the Customer interacts with the system via main.py, how the RentalSystem validates the request and implements business rules, and how the Database stores the booking information. In the approval step, there is synchronization between the booking status and the available cars.

4.4.4. Entity Relationship (ER) Diagram

ER Diagram - Data Persistence Design (SQLite)

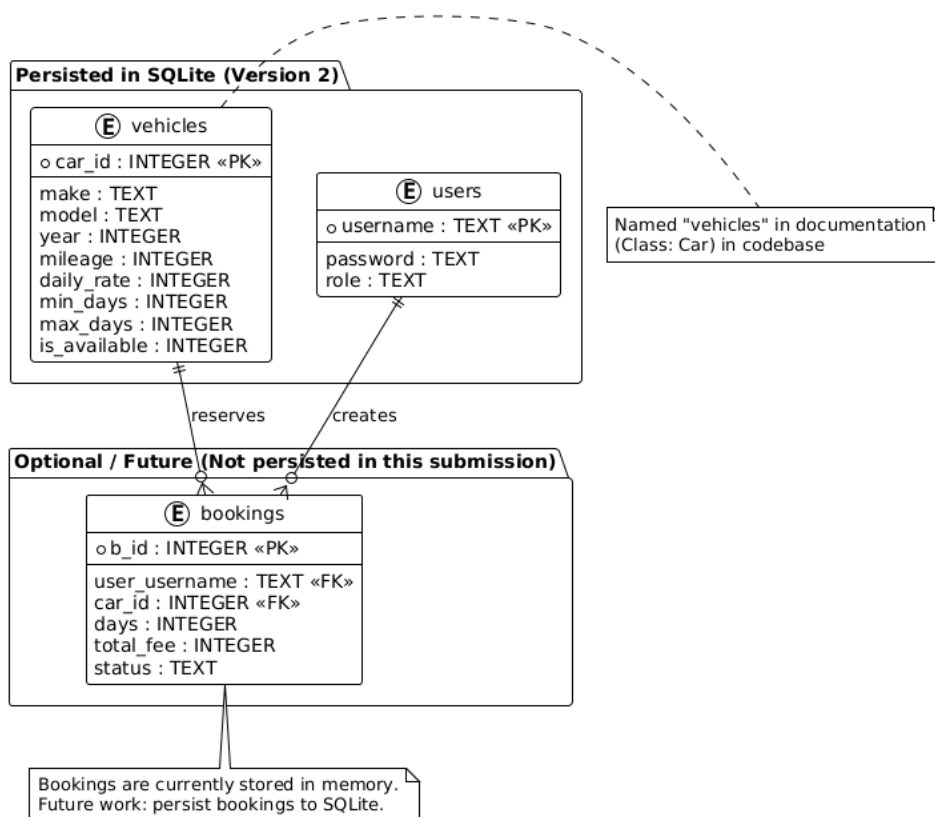


Figure 5. ER Diagram

The ER diagram defines User, Car, and Booking entities. Users and vehicles have one-to-many relationships with Bookings. This model ensures data integrity and supports future database scaling.

4.4.5 UML Summary

Together, the Class, Use Case, and Sequence Diagrams provide a comprehensive representation of the system's structure and behavior. They collectively demonstrate compliance with Task 1 requirements and support understanding of design decisions, role separation, and runtime flow.

5. Implementation and Code Walkthrough

In this section, we discuss how the Car Rental Management System was implemented using Python 3.10+. We also discuss the major components of the system and their roles. The implementation is based on the object-oriented design and architecture discussed in the previous sections of this report.

5.1 Project Modules and Responsibilities

Table 4. Project Modules and Responsibilities Table

File / Module	Purpose (Why it exists)
main.py	Presentation layer. Menu flow for login/register and role-specific actions. Keeps UI separate from business rules.
src/models.py	Entity layer. Defines User (ABC), Admin, Customer, Car, and Booking. Demonstrates OOP concepts.
src/factory.py	UserFactory (Factory Method). Validates role selection and enforces Admin Secret for secure admin creation.
src/system.py	RentalSystem (Singleton). Central coordinator for vehicle management, booking logic, and status updates.
src/database.py	SQLite data access layer. Creates tables, manages connections, and ensures persistence without manual setup.
database/rental.db	SQLite database file created automatically. Stores users, cars, and bookings.
logs/system.log	Automatic event log for auditing and maintenance troubleshooting.

requirements.txt	Confirms standard library only. Ensures compliance with submission requirements.
README.md	User documentation with installation and operational instructions.

5.2 Application Entry Point (main.py)

The main.py file acts as the **control layer** of the application. It manages:

- User input via a menu-driven Command-Line Interface (CUI)
- Role-based navigation for Admin and Customer users
- Input validation to prevent system crashes

The main loop ensures that the system remains responsive until the user explicitly exits.

Key responsibilities:

- Authentication and registration flow
- Delegation of business logic to RentalSystem
- Handling invalid inputs and user-friendly error messages

The following illustrates the control flow of the application.

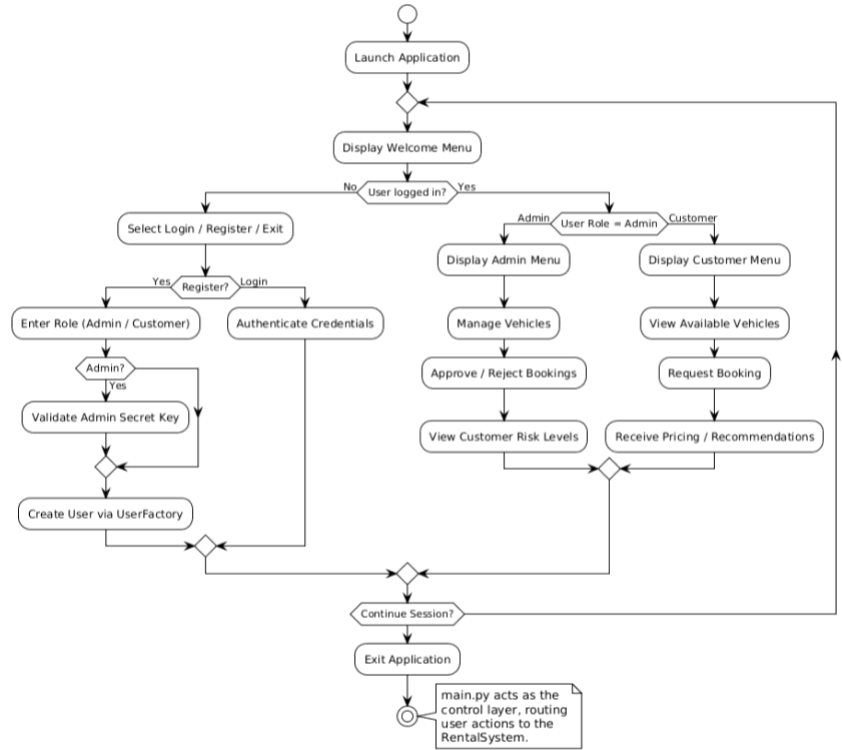


Figure 6. Application control flow

5.3 Step-by-Step Example: Customer Booking Flow

1. Run the application: `python main.py`.
2. Register/login as a customer (validated against stored users).
3. View Available vehicles (cars where `is_available` is true).
4. Book a Car by entering a valid car ID and rental days.
5. System validates availability and min/max rental constraints, then calculates fee ($\text{rate} \times \text{days}$).
6. Booking is created with status 'Pending' and stored in memory (users and vehicles are persisted via SQLite in this submission).
7. Admin logs in and reviews pending bookings, then approves or rejects.
8. If approved, car availability is updated to prevent double-booking and saved to SQLite.

```

def main(): 1 usage & rukshanm123-art *
    sys = RentalSystem()
    user = None

    while True:
        try:
            #AUTH MENU (NO USER LOGGED IN)
            if not user:
                print("\n=== YOOBEE CAR RENTAL SYSTEM ===")
                ch = input("1. Login\n2. Register\n3. Exit\n> ").strip()

                if ch == "2":
                    #Typo prevention loop for role
                    while True:
                        role = input("Register as (customer/admin): ").lower().strip()
                        if role in ["admin", "customer"]:
                            break
                        print(f"!!! '{role}' is invalid. Use 'admin' or 'customer'.")

                    un = input("Username: ").strip()
                    pw = input("Password: ").strip()
                    sec = input("Admin Secret: ").strip() if role == "admin" else ""

                    try:
                        #Store user in system memory
                        sys.users[un] = UserFactory.create_user(un, pw, role, sec)
                        print("Registration Successful!")
                    except Exception as e:
                        print(f"Error: {e}")

```

Figure 7(a): Main application loop and authentication menu logic implemented in main.py

```

        elif ch == "1":
            un = input("User: ").strip()
            pw = input("Pass: ").strip()
            u = sys.users.get(un)

            if u and u.check_password(pw):
                user = u
                print(f"Login successful. Welcome {user.get_username()}!")
            else:
                print("Login Failed.")

        elif ch == "3":
            print("Exiting... Goodbye!")
            break
        else:
            print("!!! Invalid option. Please choose 1, 2, or 3.")

    #USER MENU (LOGGED IN)

    else:
        user.display_menu()
        cmd = input("> ").strip()

        #ADMIN MENU

        if user.role == "Admin":...

        #CUSTOMER MENU

        elif user.role == "Customer":...

    except Exception as e:
        #Global protection: prevents the system from crashing due to unexpected issues
        print(f"System Error: {e}")

```

Figure 7(b): Role-based menu handling and delegation of Admin and Customer operations within the main control loop.

Figures 7(a) and 7(b) together represent the overall control flow of the application. The first part of the code is responsible for user authentication, registration, and session control, while the second part is responsible for role-based menu rendering and operation delegation to the system logic. This provides overall control, separation of responsibility, and error handling during execution.

5.4 Core Business Logic (system.py)

The RentalSystem class is the **central coordinator** of the application and follows the **Singleton design pattern**, ensuring that only one system instance exists throughout execution.

Responsibilities include:

- Managing users, vehicles, and bookings
- Enforcing rental duration constraints
- Applying dynamic pricing and discounts
- Tracking customer risk levels
- Coordinating database persistence and restoration

This approach ensures a consistent system state and prevents duplication of critical resources.

```

import logging
import os
import sqlite3

class RentalSystem: 3 usages  &rukshanm123-art
    """
    Design Pattern: Singleton
    Ensures only one instance of the rental system exists,
    centralizing data management for users, cars, and bookings.
    """
    _instance = None

    DB_PATH = "database/rental.db"

    def __new__(cls): &rukshanm123-art
        if cls._instance is None:
            cls._instance = super(RentalSystem, cls).__new__(cls)
            cls._instance.users = {}
            cls._instance.cars = []
            cls._instance.bookings = []

            #Professional Logging setup
            if not os.path.exists("logs"):
                os.makedirs("logs")
            logging.basicConfig(
                filename="logs/system.log",
                level=logging.INFO,
                format="%asctime)s - %(message)s"
            )

            #Minimal Restore: Load users/cars from SQLite (does not change booking logic)
            cls._instance._load_users_from_db()
            cls._instance._load_cars_from_db()

        return cls._instance

```

Figure 8. RentalSystem class implementation (Singleton pattern)

5.5 Object Models (models.py)

The domain model is implemented using clearly defined classes:

Table 5. Object Models Table

Class	Description
User	Abstract base class defining common user behaviour
Admin	Inherits from User; manages vehicles and bookings
Customer	Inherits from User; performs bookings
Vehicle	Represents a rentable vehicle
Booking	Represents a booking request and status

Encapsulation, inheritance, and polymorphism are applied consistently to maintain clean separation of concerns.

5.6 Database Layer (database.py)

In this submission, SQLite persistence is implemented for users and vehicles only.

Booking records are maintained in memory to simplify workflow logic, with full booking persistence identified as future work..

Stored entities include:

- Users
- Vehicles

Bookings are intentionally managed in memory to simplify workflow logic for this assignment.

```
import sqlite3
import os

class Database:
    def __init__(self, db_name="rental.db"):
        #Get project root directory safely
        base_dir = os.path.dirname(os.path.dirname(__file__))
        db_dir = os.path.join(base_dir, "database")

        #Ensure database folder exists
        if not os.path.exists(db_dir):
            os.makedirs(db_dir)

        #Create/connect SQLite database
        self.db_path = os.path.join(db_dir, db_name)
        self.conn = sqlite3.connect(self.db_path)
        self.create_tables()

    def create_tables(self):
        """Create required tables if they do not already exist"""
        cursor = self.conn.cursor()

        cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            username TEXT PRIMARY KEY,
            password TEXT NOT NULL,
            role TEXT NOT NULL
        )
        """)
```

Figure 9. SQLite table creation logic

5.7 Data Persistence Rationale (SQLite)

SQLite has been chosen for its embedded nature, reliability, and presence in the Python standard library. SQLite meets the ‘no manual configuration’ criteria and also facilitates repeated testing and persistence. The database layer will automatically create a database file and tables when it is first run.

6. Deployment and Release Build

The Car Rental Management System is designed as a **self-contained local application** requiring minimal setup.

6.1 Deployment Environment

Table 6. Deployment Environment Table

Component	Specification
Programming Language	Python 3.10+
Database	SQLite
Operating System	Cross-platform (Windows, macOS, Linux)
Dependencies	Python Standard Library only

6.2 Build and Execution Process

No compilation or external build tools are required.

Steps to run the system:

1. Extract the project directory.
2. Navigate to the root folder.
3. Execute the application using:
`python main.py`

The database and logging directories are created automatically at runtime.

6.3 Release Management

Version control is handled using Git and GitHub.

The repository contains:

- Version history (Version 1 → Version 2)
- Incremental feature enhancements
- Clear commit messages for traceability

This approach supports maintainability and collaborative development.

7. User Documentation

This section provides instructions for end users interacting with the system.

7.1 User Roles

Table 7. User Roles Table

Role	Description
Admin	Manages vehicles and approves bookings
Customer	Browses vehicles and requests bookings

7.2 Admin Workflow

Admin users can:

1. Log in using the admin secret key
2. Add, update, or remove vehicles
3. View pending booking requests
4. Approve or reject bookings
5. View customer risk levels

```
Login successful. Welcome admin!

--- ADMIN MENU (admin) ---
1. View Vehicles
2. Add New Car
3. Update Car
4. Remove Car
5. Manage Bookings
6. Logout
>
```

Figure 10. Admin menu options displayed in the CUI

7.3 Customer Workflow

Customer users can:

1. Register or log in
2. View available vehicles
3. Request a booking with rental duration
4. View booking status and pricing breakdown

When booking conditions fail, the system provides **smart alternative vehicle recommendations**.

```
--- Available Cars ---
[1] Toyota Prius - $50/day (Min:15 Max:60)
[2] Honda Civic - $40/day (Min:5 Max:30)

--- CUSTOMER MENU (user) ---
1. View Available Cars
2. Book a Car
3. View My Status
4. Logout
> 2
Car ID: 1
Days: 6
!!! Error: Rental must be between 15 and 60 days.

--- Recommended Alternatives (Closest Price) ---
[2] Honda Civic - $40/day (Min:5 Max:30)
```

Figure 11. Customer booking and recommendation output

7.4 Error Handling and Feedback

The system provides clear feedback for:

- Invalid numeric input
- Rental duration violations
- Unavailable vehicles
- Authentication failures

This ensures a smooth and user-friendly experience.

8. Testing Strategy and Test Cases

8.1 Testing Approach

The testing was done at three levels: unit testing for validation and calculations, integration testing for the interaction of modules (UI, system, database), and system testing for complete workflow scenarios for both roles.

8.2 Test Cases (Traceable to Requirements)

Table 8. Test Cases Table

TC ID	Scenario	Steps (Summary)	Expected Result	Req.	Notes
TC0 1	Register Customer	Register as customer with valid username/password	Customer created; login possible	FR1	User stored; role enforced
TC0 2	Register Admin (valid secret)	Register as admin with secret yoobee2024	Admin created; admin menu available	FR1	Prevents unauthorised role creation
TC0 3	Register Admin (invalid secret)	Register with wrong secret	Registration rejected	FR1	Security validation

TC0 4	Login success	Login with valid credentials	Correct role menu displayed	FR1	Polymorphic menu
TC0 5	Login failure	Login with incorrect password	Login fails; system remains stable	FR1	No crash
TC0 6	Admin add car	Add car with valid fields	Car added and shown in vehicles	FR2	Car persisted/restored across restart
TC0 7	Admin update mileage	Update mileage for existing ID	Mileage updated	FR2	Validation + logging
TC0 8	Admin remove car	Remove car by ID	Car removed from vehicles	FR2	Car record removed from persistence (if enabled)
TC0 9	View available vehicles (empty)	Customer views vehicles when none are available	No available vehicles message	FR3	Usability
TC1 0	Book car invalid ID	Book non-existent car ID	Rejected with error message	FR3	Reliability
TC1 1	Book car outside min/max	Try days < min or > max	Rejected with guidance message	FR3	Validation
TC1 2	Book car successful	Valid car ID + valid days	Pending booking created; pricing breakdown displayed	FR3/FR6	Transparent output
TC1 3	Discount applied (≥ 7 days)	Book for 7+ days within limits	5% discount reflected in breakdown	FR4/FR6	V2 feature

TC1 4	Discount applied (≥ 14 days)	Book for 14+ days within limits	10% discount reflected in breakdown	FR4/FR 6	V2 feature
TC1 5	Recommendation s on booking failure	Attempt invalid booking (unavailable/invalid duration)	Alternative vehicles suggested (excluding selected car)	FR7	V2 feature + filtering
TC1 6	Admin approve booking	Approve pending booking	Status Approved; car becomes unavailable	FR5	Prevents double- booking
TC1 7	Admin reject booking	Reject pending booking	Status Rejected; car remains available	FR5	Correct status
TC1 8	Risk indicator shown to Admin	Create rejected/pending booking then view manage bookings	Risk level displayed for customer	FR8	Decision support
TC1 9	SQLite restore after restart	Restart app after adding users/cars	Users/ vehicles restored from SQLite	FR9	Bookings may not persist
TC2 0	Logging created	Perform key actions	logs/system.log contains entries	NFR	Audit trail

8.3 Coverage Discussion

The test suite covers all functional requirements (FR1–FR9) and key non-functional requirements. Boundary testing was included for invalid IDs, invalid durations, and invalid numeric inputs to demonstrate robustness.

9. Innovative Solution (Task 2 – LO2)

9.1 Industry Need

Car rental demand varies by season, weekends, and events. A static pricing strategy may result in lost sales in peak demand periods and underutilization in low demand periods. A pricing recommendation based on data can enhance decision-making and build a competitive edge.

9.2 Proposed Innovation: AI-Driven Demand Forecasting for Dynamic Pricing

Another future enhancement could include an AI-based demand forecasting module. With the booking history stored in SQLite, the system could potentially forecast demand and suggest pricing changes (e.g., increase prices for peak periods, offer discounts for long rentals during low periods).

9.3 Architectural Support

- Future work can persist booking history in SQLite to enable analytics and forecasting, enabling training datasets.
- Layered design allows the forecasting module to be added without changing the CUI layer.
- A Strategy pattern can later provide pluggable pricing algorithms (seasonal, demand-based, loyalty-based).

10. Software Evolution, Maintenance, and Backward Compatibility (Task 3 – LO2)

10.1 Maintenance Plan

The Car Rental Management System was developed through an iterative approach in software engineering, where the system was built from an initial baseline (Version 1) to an advanced and intelligent version (Version 2). This shows an understanding of software maintenance, flexibility, and system sustainability.

Corrective Maintenance

Corrective maintenance is the maintenance carried out on defects that are identified during the testing and execution phase. In the given system, corrective maintenance involved correcting defects in the role initialization in the User inheritance hierarchy, correcting defects in the input validation cases, and ensuring that invalid numeric inputs such as symbols are not accepted by the system.

Adaptive Maintenance

Adaptive maintenance was carried out to meet new requirements and adapt to changing environments. The inclusion of SQLite for the restoration of user and vehicle data can be classified as an adaptive maintenance activity, which provides the system with the ability to ensure continuity between executions. In addition, some terminological improvements, such as changing “fleet” to “vehicles,” were made to improve the system's readability and consistency with terminological usage in the field.

Perfective Maintenance

Perfective maintenance activities were carried out with the aim of improving the usability and functionality of the system, going beyond the basic requirements. In Version 2, improvements were made to the system's usability and functionality. For instance, the inclusion of dynamic pricing and long-term rental discounts, detailed pricing, smart alternatives, and risk management were made to improve the system's usability and functionality.

Backward Compatibility Strategy

In Version 2, all improvements were made in such a manner that did not alter the basic structure of the system. In addition, no improvements were made that would compromise the existing functionality. Behavioral extensions were made by adding new methods to existing classes.

10.2 Versioning Strategy

Semantic versioning (MAJOR.MINOR.PATCH) is used. v1.0.0 is the initial version, v1.1.0 includes non-breaking enhancements, and v2.0.0 includes breaking changes such as a major CLI redesign or schema changes.

10.3 Versioning History

Version 1 – Baseline Implementation

The first version of the system was built to meet the essential functional requirements of the system. These functional requirements include role-based authentication, vehicle management (CRUD operations), booking requests, booking approval, and input validation. The system was built by applying object-oriented concepts and object-oriented design patterns such as the Singleton and Factory Method design patterns.

Version 2 – Enhanced Intelligent System

The enhanced system was built by adding more sophisticated and advanced business logic to the original system. The additional features of the enhanced system include dynamic pricing with discounts for long-term rentals, transparent pricing information, intelligent alternative vehicle suggestions, risk-based booking management for Admin users, and the ability to restore user and vehicle information using SQLite. The enhanced system represents the evolution of the software by adding features to the original system.

10.4 Backward Compatibility Strategy

Schema changes should be additive as much as possible (add new nullable columns, etc.). For non-additive changes, migration scripts should be used to transform existing data to fit the new schema. Deprecations should be maintained for at least one minor version cycle.

11. User Documentation Summary

Detailed instructions are found in the README.md file. In brief, it is necessary to install Python 3.10+, execute the following line in the terminal: ``python main.py``, register/login, and use the role-specific menu options. The system will create the database/ directory and logs/ directory automatically if necessary.

12. Limitations and Future Work

Known Limitations

The system was designed with limitations based on its intended use as an academic assignment. For instance, the system stores users' passwords in plain text. However, it was not within the scope of the assignment to harden the system. Moreover, the system does not store booking records. These records are kept in memory. Finally, the system is not designed to have a user interface other than the command line.

Future Enhancements

There are several possible additions to the system that could make it better. For instance, it could have an encryption algorithm for users' passwords. In addition, it could have an analytics system to analyze booking trends. However, the current system architecture allows for the inclusion of AI algorithms to forecast demand. Moreover, it could have a user interface other than the command line.

13. Conclusion

This project successfully proves the design and implementation of a Car Rental Management System with the application of object-oriented programming concepts and good software development principles. Through a series of iterations, the system has moved from a basic working system to a smart system with the application of dynamic pricing, decision support, and data persistence mechanisms.

The application of design patterns, modular design, and strong validation mechanisms has led to a robust and scalable system design. This project has followed good software development principles, including system evolution, backward compatibility, and long-term growth potential, making it a fully working system for the assignment requirements.

14. References

Python Software Foundation. (2024). *Python 3 documentation*. <https://docs.python.org/3/>

SQLite Consortium. (2024). *SQLite documentation*. <https://www.sqlite.org/docs.html>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Object Management Group. (2023). *Unified Modeling Language (UML) specification*. <https://www.omg.org/spec/UML/>

De Silva, R. (2026). *Car rental management system (CUI)* [Source code]. GitHub. <https://github.com/rukshanm123-art/mse800-car-rental-system>